The Journal of Open Source Software

SLAP2 Utils: Tools for Processing SLAP2 Data

Peter Hogg[1*¶], **Shijie Jerry Tong**[1*], **and Kurt Haas**[1]

1 Department of Cellular and Physiological Sciences, Centre for Brain Health, School of Biomedical Engineering, University of British Columbia, Vancouver, Canada ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 22 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Two-photon microscopy enables the measurements of neural activity in vivo, deep within the brain using fluorescent biosensors. However, capturing neural activity with voltage indicators, fast calcium or neurotransmitter biosensors, or slower indicators across 3D volumes requires sampling speeds that far exceed capabilities of conventional laser-scanning two-photon microscopes. The recently developed Scanned Line Angular Projection-2 (SLAP2) microscope has overcome these limitations by employing novel ultra-fast random-access scanning techniques to record neural activity at kilohertz sampling rates. Unlike traditional raster imaging, where pixels are recorded sequentially by spatial position, SLAP2's random-access acquisition optimizes the sampling order of regions of interest for speed. These recordings are stored in a complex file structure, which tracks the non-sequentially sampled coordinates. This novel data structure is incompatible with post-imaging analysis tools designed for traditional image formats, such as TIF or CZI files. Here, we present our Python library, `SLAP2 Utils`, to interact with the custom data structure generated from SLAP2. `SLAP2 Utils` allows users to extract neuronal activity directly from the SLAP2 binary files in a Python environment. Additionally, we provide extensions to the SLAP2 microscope control and image acquisition software, providing a framework for user-designed microscope functions during in vivo experiments.

Statement of need

To overcome the inherent speed limitations of laser scanning two-photon imaging, the Scanned Line Angular Projection (SLAP) microscope was developed using projection microscopy and deconvolution to sample neuronal activity in kilohertz rates (Kazemipour et al., 2019). The second generation of this technology, called SLAP2, builds upon this technology for random-access sampling of regions of interest (ROIs) throughout 3D volumes without deconvolution and is commercially available as a kit (Podgorski, 2021). SLAP2 conducts random-access imaging of preassigned ROIs during high-speed acquisitions by transitioning between ROIs with an inertia-free scan engine. However, this results in a complex scan pattern as the microscope can sample ROI that are not adjacent to each other, complicating the task of indexing the raw recording. Currently, the custom tools needed to read data from the binary files generated during high-speed acquisitions are written using proprietary programming languages, making it cumbersome to design analysis workflows that leverage packages in the Python ecosystem. To overcome this limitation, we developed the Python library `SLAP2 Utils`. Our package provides pure Python implementations of the functions needed to read raw SLAP2 microscope data, facilitating Python-based analysis pipelines. Additional functionalities include visualizing recorded data, core processing functions, and extensions of the primary SLAP2 software used during experiments.

DRAFT

SLAP2 Datafile Pipeline

SLAP2 `Utils` provides an open-source version of Matlab functions (Podgorski, 2021) designed to interface with binary files generated from high-speed recordings of neural activity from the SLAP2 two-photon microscope. This implementation uses object-orientated programming (OOP), creating Python Classes for parsing out the data for analytical pipelines (see Figure 1) (Podgorski, 2021). This is needed because, unlike raster imaging that the SLAP2 microscope can also preform, which generates TIF formatted digital images, high-speed random-access imaging produces two files: a binary file (.dat) with an accompanying metadata (.meta) file that stores acquisition information using the HDF5 format. The binary file consists of a header, which stores core acquisition settings in unsigned 32-bit integers, followed by the digitized recording of the microscope's detectors in unsigned 16-bit integers. The `MetaData` file contains more detailed information about the acquisition, including key parameters, such as the `ParsePlan`, a collection of data describing the timing of axial focus and coordinates being sampled by the microscope. This information about the acquisition is needed to index the recording in the binary file. During random-access imaging, the microscope will sample ROIs at different acquisition rates in an order determined by a planning algorithm, which may not be in any particular spatial order. Given the complications in random-access imaging and the organization of the recording in the binary file, a simple linear readout of the DAT file's contents would be insufficient to extract the neural activity. The core function of our Python library handles this task with our `DataFile` object that parses raw data and meta-data stored in these files directly into a Python environment. This `DataFile` class, upon initialization, loads information from the file header of the binary file and creates a subclass, MetaData, which will load additional acquisition information using the h5py library (Collette, 2013). The digitized signals of the acquisition are not loaded into memory. Rather, the Datafile class generates a memory map (memap) of the 16-bit raw recording in the binary file using Numpyr(Harris et al., 2020). This memap is used to index specific data in the raw recording when needed, helping end users work with larger datasets collected at kilohertz acquisition rates.

To further aid with retrieving traces of neural activity from the random-access acquisitions, we implemented an additional Python class, `Trace`, which streamlines this operation. This class initializes with three variables: a `DataFile` object paired with the z-axis and channel ID indices. Before the `Trace` object can generate an activity trace, users must first call the `Trace.setPixelIdxs()` method. This method uses boolean arrays representing areas in the field of view (FOV). The `Trace` object uses these coordinates to generate indices in the `DataFile`'s memap during trace extraction. The boolean arrays can be generated from SLAP2 ROI objects stored in the `MetaData` class, allowing the `Trace` object to generate activity profiles from individual ROIs. Once these inputs are finalized, the `Trace.process()` method will return an activity trace to the user. Additionally, each ROI is composed of smaller units, superpixels, and information for each superpixel is stored in a subclass `TracePixel` accessible through the `Trace.TracePixels` attribute in the parent class. This allows for higher resolution interrogation of the spatial domains of neuronal activity within a ROI; it should be noted that while `Trace.TracePixels` is a list; it is not ordered spatially until the `Trace.orderadjust()` method is called.

DRAFT

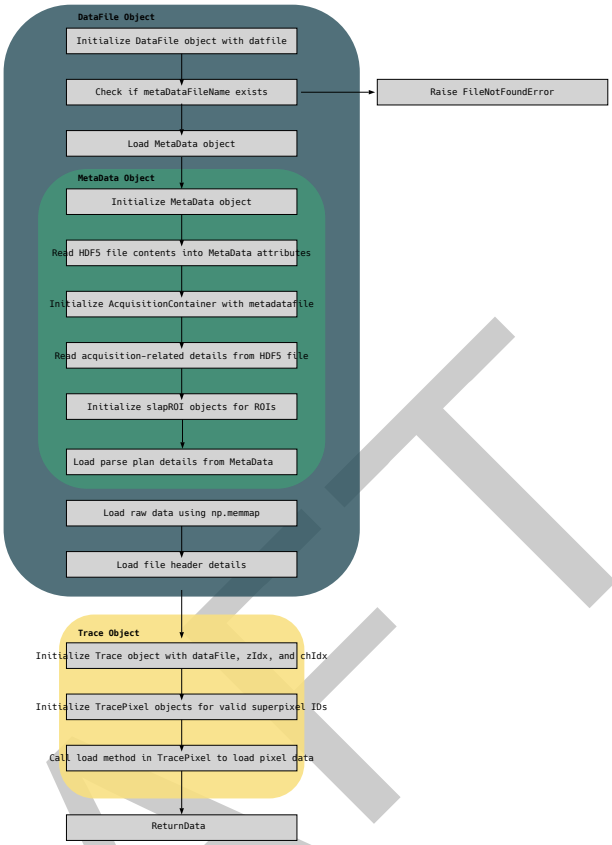

Figure 1: Figure 1: SLAP2 Pipeline. The DataFile class is initialized with the path to a .dat file. The MetaData subclass will be initialized if the .meta file is found. A DataFile object is then used to initialize the Trace Class, which has methods for easy data extraction from the binary file.

SLAP2 Uility UI that Accompanies SLAP2 Software

In addition to the Python library, extensions to the core SLAP2 software are packaged in the software repository. This code combines MATLAB and Python utilities controlled by a custom user interface (UI). The script directly interfaces with objects in the SLAP2 software, primarily written in MATLAB. The UI includes valuable functions such as generating ROIs automatically from reference images, creating a reference stack, and updating ROI positions due to sample drift Walt et al. (2014). Interfacing directly with the SLAP2 software allows these scripts to automate the workflow during an experiment, minimizing human error that can arise during *in vivo* imaging sessions. The calculation of the shift adjustment used to update ROI positions uses phase cross correlation between the live view of the sample and a reference stack. Our implementation of this calculation is a GPU-accelerated rewrite of the algorithm used in the `scikit-image` library using `cupy` [Kuglin & Hines (1975), Walt et al. (2014)](Okuta et al., 2017). These GPU-accelerated speed-ups allow the function to perform in real-time.

SLAP2 Utility Functions and Visualizations

Several commonly used functions are included for tasks that would be routinely called upon. These utility functions simplify generating the traces from ROIs, cleaning up traces from volumetric recordings, and extracting ROI information in both 2D and 3D. Additional utility functions interact with other file types generated by the microscope, such as extracting stimulus times from stimulus logs stored in H5 files and functions for averaging reference stacks stored

DRAFT

as TIF files. Useful plotting functions are also packaged in the library, such as a function that plots ROIs over a reference image. Furthermore, the package comes with a Python-based data viewer that allows for the inspection of activity and ROIs from a given DAT file with a GUI. The data viewer's features include 3D inspection of ROIs over a reference volume and interactive plots of activity traces.

Availability

All the code and scripts mentioned above are publicly available on the GitHub repository ([https://github.com/Peter-Hogg/SLAP2_Utils/tree/main]). The Python library can be installed using pip.

Acknowledgements

The development of this Python library was supported by funds from the Canadian Institutes of Health Research (CIHR) Foundation Award (FDN-148468) and NSF Award 2019976, "AccelNet: International network for brain-inspired computation".

References

Collette, A. (2013). *Python and HDF5*. O'Reilly.

Harris, C. R., Millman, K. J., Walt, S. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Rio, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Kazemipour, A., Novak, O., Flickinger, D., Marvin, J. S., Abdelfattah, A. S., King, J., Borden, P. M., Kim, J. J., Al-Abdullatif, S. H., Deal, P. E., Miller, E. W., Schreiter, E. R., Druckmann, S., Svoboda, K., Looger, L. L., & Podgorski, K. (2019). Kilohertz frame-rate two-photon tomography. *Nature Methods*, *16*(8), 778–786. https://doi.org/10.1038/s41592-019-0493-9

Kuglin, C., & Hines, D. C. (1975). The phase correlation image alignment method. *Proceeding of IEEE International Conference on Cybernetics and Society*, 163–165.

Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible library for NVIDIA GPU calculations. *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-First Annual Conference on Neural Information Processing Systems (NIPS)*. http://learningsys.org/nips17/assets/papers/paper_16.pdf

Podgorski, K. (2021). SLAP2 – two photon microscope kit. In *Product Page*. MBF Bioscience. https://www.mbfbioscience.com/products/slap2

Walt, S. van der, Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & contributors, the scikit-image. (2014). Scikit-image: Image processing in python. *PeerJ*, *2*(453). https://doi.org/10.7717/peerj.453

DRAFT